What MAG can see, and what it cannot

Generated by `engine/mechanics_coverage.js`*. Do not hand-edit — re-run it.*

Format `gen9championsvgc2026regmb`. Every ability on a species with non-zero usage, weighted by usage share, so a rare ability on a common Pokémon outranks a common one nobody brings.

How this is measured

By experiment, not by searching source. Each channel is exercised by running the real code twice — once with the ability, once with a neutral control — and comparing the output. If nothing moves, the model cannot see it.

The previous version of this document grepped for ability names and was wrong in both directions: it credited `speedboost` and `prankster` to the damage engine because they appear in a species lookup table, and it missed twelve type-immunity abilities because that table is written with unquoted keys. **Those numbers should not be quoted.**

Summary

	count	usage share
abilities in this format	192	100%
MAG responds to it	**35**	**34.47%**
blind — has dex handlers, no response	**155**	**59.36%**
no handlers (nothing to model)	2	0.04%

Blind spots, ranked by what they cost

"Blind" means MAG cannot ANTICIPATE the ability, not that it never sees the consequence. That distinction matters and the list overstates without it. Intimidate is `onStart`: once it has fired, `board.js` reads the -1 Attack from the protocol like any other stat stage, so MAG sees the *result*. What it cannot do is know that bringing that Pokémon in will cause it. Weather setters are the same shape — `board.weather` is tracked once the weather is up, but nothing knows that switching Pelipper in sets rain, or that rain then doubles a Swift Swim partner's Speed.

That is why these are concentrated in `onStart` and `onSwitchInPriority`: they are the **conditional, one-step-ahead** mechanics, and a static feature vector describing the board as it stands cannot represent any of them. Weather setters alone are 9.4% of the format (`drought` 3.40 + `drizzle` 3.24 + `snowwarning` 1.82 + `sandstream` 0.93) and Intimidate another 5.65%. No amount of feature work reaches them; a search that plays the switch out does, because the simulator applies them for free.

ability	usage share	what it does (dex handlers)
intimidate	5.65%	onStart
hospitality	5.42%	onSwitchInPriority, onStart
roughskin	4.51%	onDamagingHitOrder, onDamagingHit
defiant	3.68%	onAfterEachBoost
drought	3.40%	onStart
drizzle	3.24%	onStart
toughclaws	2.91%	onBasePowerPriority, onBasePower
unburden	2.26%	onAfterUseItem, onTakeItem, onEnd
stamina	2.10%	onDamagingHit
pixilate	2.02%	onModifyTypePriority, onModifyType, onBasePowerPriority, onBasePower
snowwarning	1.82%	onStart
fairyaura	1.52%	onStart, onAnyBasePowerPriority, onAnyBasePower
competitive	1.10%	onAfterEachBoost
sandstream	0.93%	onStart
friendguard	0.87%	onAnyModifyDamage
speedboost	0.83%	onResidualOrder, onResidualSubOrder, onResidual
unnerve	0.83%	onSwitchInPriority, onStart, onEnd, onFoeTryEatItem
regenerator	0.71%	onSwitchOut
noguard	0.61%	onAnyInvulnerabilityPriority, onAnyInvulnerability, onAnyAccuracy
innerfocus	0.58%	onTryAddVolatile, onTryBoost
unaware	0.58%	onAnyModifyBoost
shadowtag	0.57%	onFoeTrapPokemon, onFoeMaybeTrapPokemon
liquidvoice	0.55%	onModifyTypePriority, onModifyType
scrappy	0.53%	onModifyMovePriority, onModifyMove, onTryBoost
magicbounce	0.50%	onTryHitPriority, onTryHit, onAllyTryHitSide
poisontouch	0.49%	onSourceDamagingHit
moldbreaker	0.46%	onStart, onModifyMove
sharpness	0.44%	onBasePowerPriority, onBasePower
mirrorarmor	0.43%	onTryBoost
megalauncher	0.41%	onBasePowerPriority, onBasePower
clearbody	0.41%	onTryBoost
protean	0.39%	onPrepareHit
frisk	0.37%	onStart

ability	usage share	what it does (dex handlers)
`stancechange`	0.36%	onModifyMovePriority, onModifyMove
`flamebody`	0.33%	onDamagingHit
`sheerforce`	0.32%	onModifyMove, onBasePowerPriority, onBasePower
`toxicdebris`	0.30%	onDamagingHit
`spicyspray`	0.30%	onDamagingHit
`megasol`	0.27%	onWeatherModifyDamagePriority, onWeatherModifyDamage
`rockhead`	0.26%	onDamage
…and 115 more		

What MAG does respond to

ability	usage share	channels
prankster	9.04%	priority
adaptability	3.95%	damage
levitate	3.40%	blocks, damage
armortail	3.00%	blocks, side-wide
swiftswim	2.53%	speed
goodasgold	1.88%	blocks
hugepower	1.63%	damage
lightningrod	1.30%	damage
chlorophyll	0.83%	speed
galewings	0.83%	priority
contrary	0.77%	boosts
technician	0.72%	damage
flashfire	0.63%	blocks, damage
multiscale	0.62%	damage
thickfat	0.57%	damage
sandrush	0.46%	speed
queenlymajesty	0.44%	blocks, side-wide
oblivious	0.34%	blocks
waterbubble	0.24%	damage
soundproof	0.18%	blocks
dryskin	0.16%	blocks, damage
voltabsorb	0.15%	blocks, damage
eelevate	0.12%	blocks, damage
filter	0.11%	damage
solarpower	0.11%	damage
bulletproof	0.09%	blocks
eartheater	0.08%	blocks, damage
purifyingsalt	0.06%	blocks
solidrock	0.06%	damage
sapsipper	0.04%	damage
purepower	0.04%	damage

ability	usage share	channels
waterabsorb	0.04%	damage
heatproof	0.03%	damage
slushrush	0.02%	speed
motordrive	0.02%	damage

Items

item	usage share	responds via
focussash	12.05%	**nothing**
sitrusberry	10.37%	*berry — not modelled*
lifeorb	8.06%	damage
leftovers	6.73%	**nothing**
choicescarf	6.52%	speed
lightclay	3.24%	**nothing**
staraptite	2.72%	mega forme
whiteherb	2.24%	**nothing**
chopleberry	2.07%	*berry — not modelled*
swampertite	2.02%	mega forme
fairyfeather	1.95%	**nothing**
metagrossite	1.94%	mega forme
colburberry	1.85%	*berry — not modelled*
kasibberry	1.77%	*berry — not modelled*
raichunitey	1.73%	mega forme
charizarditey	1.64%	mega forme
charcoal	1.53%	**nothing**
floettite	1.52%	mega forme
mawilite	1.38%	mega forme
roseliberry	1.34%	*berry — not modelled*
mysticwater	1.31%	**nothing**
blackglasses	1.30%	**nothing**
mentalherb	0.99%	**nothing**
sceptilite	0.81%	mega forme
widelens	0.80%	**nothing**

What this does and does not say

- A response means the mechanic **reaches a number MAG uses**. It does not mean the number is correct, only that the ability is not invisible. `engine/feature_coverage.js` answers the different question of whether a feature ever fires in real games.
- The damage probe varies weather, terrain, move type, category, contact/slicing/bullet flags, attacker and defender, and two HP states. An ability needing a condition outside that grid would be reported blind when it is not. That is the remaining known bias and it points toward **over**-reporting blindness, which is the safe direction for a gap list.
- Usage shares are of the whole format, so a filtered table does not sum to 100%.